

Automaton Specification Language Documentation

Automaton Emulator

1 Introduction

The Automaton Specification Language (ASL) is a domain-specific language designed to define finite automata for simulation. It supports both Deterministic Finite Automata (DFA) and Non-deterministic Finite Automata (NFA).

2 File Structure

An ASL file is divided into mandatory sections, each denoted by a header in square brackets. Sections can appear in any order, though the following order is conventional:

1. `[alphabet]`
2. `[states]`
3. `[initial_state]`
4. `[accept_states]`
5. `[transitions]`

3 Sections

3.1 Alphabet

The `[alphabet]` section defines the set of symbols accepted by the automaton. Symbols are space-separated and can span multiple lines.

3.2 States

The `[states]` section lists all possible states in the automaton. Like the alphabet, states are space-separated.

3.3 Initial State

The `[initial_state]` section must contain exactly one state from the defined states list. This represents the starting point of the simulation.

3.4 Accept States

The `[accept_states]` section lists the states that lead to an "Accepted" result if the simulation ends in one of them.

3.5 Transitions

The `[transitions]` section defines the state transition function.

4 Transition Syntax

4.1 DFA Transitions

For Deterministic Finite Automata, each transition must follow the strict format:

```
1 source_state, symbol -> next_state
```

4.2 NFA Transitions

Non-deterministic Finite Automata support additional syntax for flexibility:

Single Transition:

```
1 q0, a -> q1
```

Multiple Next States:

```
1 q0, a -> q1, q2
```

Multiple Input Symbols:

```
1 q0, (a, b) -> q1
```

5 Lexical Rules

- **Comments:** Any text following a # symbol on a line is ignored.
- **Whitespace:** Leading and trailing whitespace on lines is ignored. Empty lines are ignored.
- **Case Sensitivity:** Section headers are case-insensitive ([ALPHABET] is the same as [alphabet]).

6 Example

```
1 [alphabet]
2 a b
3
4 [states]
5 s0 s1
6
7 [initial_state]
8 s0
9
10 [accept_states]
11 s1
12
13 [transitions]
14 s0, a -> s1
15 s1, b -> s1
16 s1, a -> s0
```

Listing 1: Example DFA file